DO TEXT-TO-SQL MODELS RELY ON SCHEMA SEMANTICS? A STUDY OF LEXICAL VS. STRUCTURAL INFORMATION UNDER SCHEMA ANONYMIZATION

Fengze Guo **6882479** **fengze.guo@student.uni-tuebingen.de**

1 INTRODUCTION

Large Language Models (LLMs) have recently achieved strong performance on Text-to-SQL benchmarks such as Spider (Yu et al., 2018), often reaching high execution accuracy. However, it is not always clear whether this success reflects genuine relational reasoning over database structure or is mainly driven by surface-level lexical alignment between natural language queries and schema tokens. This distinction matters for evaluating the robustness and generalization of Text-to-SQL systems, especially in settings where schema semantics are noisy, ambiguous, or unavailable.

This study examines the extent to which Text-to-SQL models rely on lexical schema semantics or relational structure. To test this distinction, I remove the lexical information from database schemas while preserving the underlying relational topology, including foreign key relationships. The research question is as follows:

> *Can Text-to-SQL models generate correct queries using structural information alone, or do they fundamentally depend on schema semantics?*

To answer this question, I use the `gemini-3-flash-preview` model and a controlled experimental setup based on schema anonymization. Two anonymization schemes are considered: (1) full anonymization, which removes all lexical information from table and column names, and (2) partial anonymization, which preserves limited semantic cues. Model performance is evaluated across multiple anonymization levels on a subset of the Spider benchmark.

The results suggest that current Text-to-SQL models rely heavily on lexical schema semantics, and struggle to generate correct queries when such information is removed.

2 BACKGROUND

A challenge in Text-to-SQL is *schema linking*, which refers to aligning natural language expressions with database elements such as tables and columns (Guo et al., 2019). Early work addressed this problem using explicit schema representations and relation-aware encodings (Wang et al., 2020), allowing models to capture structural dependencies within the database.

The Spider benchmark (Yu et al., 2018) is widely used for evaluating Text-to-SQL systems. It covers multiple domains and includes a range of query types, such as simple, join, aggregation, and nested queries. This diversity makes it suitable for analyzing how different query structures depend on schema information.

More recently, LLMs have shown strong performance on Text-to-SQL tasks without relying on explicit schema modeling (Pourreza & Rafiei, 2023; Cheng et al., 2023). In these studies, schema linking is handled more implicitly through pretraining and in-context learning, rather than through explicit structural representations. Instead of relying on explicit relations in the schema, models tend to depend more on patterns learned from data, including lexical associations between queries and schema elements.

3 Data and Methodology

3.1 Dataset

Experiments are conducted on a subset of the Spider benchmark. To keep the analysis manageable while maintaining diversity, 80 examples are sampled from the Spider development set. The subset covers a range of query types, including simple, aggregation, join, join-aggregation, and nested queries.

Each example is annotated with a query type label to support more detailed analysis. The dataset is approximately balanced across query categories, which allows comparison of model behavior under different structural conditions. The selected examples are from multiple databases and query types. Table 1 summarizes the distribution of query types in the dataset.

Query Type	Count
Simple	12
Aggregation	16
Join	16
Join-Aggregation	20
Nested	16
Total	80

Table 1: Distribution of query types in the sampled dataset.

3.2 Schema Anonymization

To study the roles of lexical semantics and relational structure, I construct a set of schema variants by progressively removing lexical information while keeping the underlying relational topology unchanged.

I apply two anonymization schemes:

- **Partial Anonymization:** Lexical cues are weakened but not completely removed, for example through abbreviations or semantically related renamings.
- **Full Anonymization:** All lexical information in table and column names is removed, leaving only structural information.

Within each scheme, four levels of schema representation are defined, corresponding to increasing degrees of lexical information removal:

- **L0 (Original Schema):** The original schema with full table and column names.
- **L1 (Abbreviated Schema):** Shorten table and column names (e.g., `name` → `nm`, `country` → `coun`), preserving partial lexical signals.
- **L2 (Typed Schema):** Replace table names with generic placeholders (e.g., `table_1`), and replace column names with type-based identifiers (e.g., `text_1`, `num_1`, `date_1`, `pk_1`, `fk_1`). This removes lexical semantics while preserving coarse type information and relational structure.
- **L3 (Fully Anonymized Schema):** Replace both table and column names with generic identifiers (e.g., `table_1`, `col_1`), removing all lexical and type information while preserving only relational structure, including foreign key relationships.

Both schemes are applied across the same levels (L0–L3), allowing direct comparison under controlled conditions. In full anonymization, lexical information is removed progressively across levels, while partial anonymization retains limited semantic cues through weaker transformations.

In all settings, the database content and relational structure (e.g., foreign key relationships) are unchanged. This setup separates the effect of lexical schema semantics from structural information and allows their roles in Text-to-SQL performance to be examined.

3.3 Prompting Setup

For each example, a prompt is constructed using the database schema, its foreign key relationships, and the natural language question. The model is instructed to generate a single executable SQL query without additional explanation.

The model is `gemini-3-flash-preview`, accessed via the official API. All generation settings are kept fixed across anonymization conditions, including the prompt format and decoding configuration. No changes are made to prompting strategy or sampling parameters across different schema variants.

3.4 Evaluation Metrics

Model performance is evaluated using two standard metrics commonly used in Text-to-SQL benchmarks such as Spider:

- **Exact Match Accuracy:** Whether the predicted SQL query exactly matches the reference query.
- **Execution Accuracy:** Whether the predicted SQL query produces the same result as the reference query when executed on the database.

Execution accuracy is used as the primary metric, as it reflects semantic correctness even when syntactic forms differ.

3.5 Analysis Setup

In addition to overall performance, I analyze results for two dimensions: (1) performance is broken down by query type to examine how different structures are affected by schema anonymization; (2) qualitative analysis is conducted on representative examples across anonymization levels and schemes. This analysis helps identify both performance trends and common failure modes, such as errors in schema grounding and the preservation of structural query patterns.

4 Results

4.1 Overall Performance

Table 2 summarizes overall performance across anonymization levels. Under L0, execution accuracy is similar for both schemes (0.912). As lexical information is reduced, performance drops sharply.

Under partial anonymization, execution accuracy decreases to around 0.050 and remains stable across L1–L3. In contrast, under full anonymization, execution accuracy drops to 0.075 at L1 and reaches zero at L2 and L3.

Exact match accuracy follows a similar trend, remaining low even under L0 and approaching zero as anonymization increases.

	L0		**L1**		**L2**		**L3**	
Scheme	Exact	Exec	Exact	Exec	Exact	Exec	Exact	Exec
Partial	0.138	0.912	0.037	0.050	0.013	0.050	0.037	0.050
Full	0.125	0.912	0.025	0.075	0.000	0.000	0.000	0.000

Table 2: Overall performance across anonymization levels. Execution accuracy uses strict matching.

As lexical information is reduced, performance drops sharply. Under partial anonymization, execution accuracy decreases to around 5% and remains at this level across L1–L3. Under full anonymization, execution accuracy decreases further and reaches zero at L2 and L3.

Exact match accuracy shows a similar trend but remains low across all settings, including L0.

4.2 Performance by Query Type

Table 3 and Table 4 show performance by query type under the two anonymization schemes.

Across both schemes, most query types drop to zero once lexical information is degraded. Simple, join, and nested queries all reach zero accuracy beyond L0.

Aggregation queries exhibit a different pattern. Under partial anonymization, they retain stable non-zero performance (0.250) across L1–L3. Under full anonymization, aggregation achieves 0.312 at L1, but drops to zero at L2 and L3. A small number of join-aggregation queries remain correct at L1 (0.050), but this does not persist at higher levels.

Query Type	L0	L1	L2	L3
Simple	0.917	0.000	0.000	0.000
Aggregation	0.938	0.250	0.250	0.250
Join	1.000	0.000	0.000	0.000
Join-Aggregation	0.800	0.000	0.000	0.000
Nested	1.000	0.000	0.000	0.000

Table 3: Execution accuracy by query type under partial anonymization.

Query Type	L0	L1	L2	L3
Simple	0.917	0.000	0.000	0.000
Aggregation	0.938	0.312	0.000	0.000
Join	1.000	0.000	0.000	0.000
Join-Aggregation	0.750	0.050	0.000	0.000
Nested	1.000	0.000	0.000	0.000

Table 4: Execution accuracy by query type under full anonymization.

Across both schemes, most query types show a complete drop in performance once lexical information is degraded. Simple, join, and nested queries all reach zero accuracy beyond L0.

Aggregation queries exhibit a different pattern. Under partial anonymization, they retain stable non-zero performance (0.250) across L1–L3. Under full anonymization, aggregation achieves 0.312 at L1, but drops to zero at L2 and L3. A small number of join-aggregation queries also remain correct at L1 under full anonymization (0.050), but this does not persist at higher levels.

Figure 1 shows the same pattern across query types. Under full anonymization (left), accuracy drops to zero beyond L1 for all categories. Under partial anonymization (right), only aggregation queries retain limited performance, while others collapse to zero.

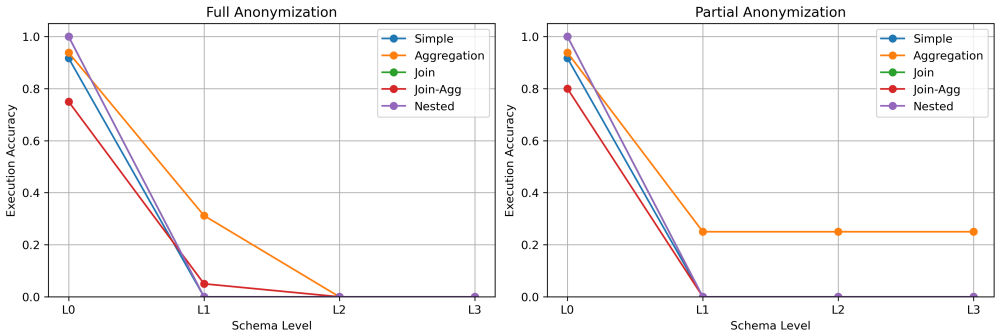

Figure 1: Execution accuracy across schema anonymization levels for different query types under full (left) and partial (right) anonymization schemes.

Overall, performance transitions from high accuracy to complete failure as lexical information is removed.

4.3 Qualitative Observations

Under partial anonymization, errors are primarily caused by incorrect or incomplete schema linking. Generated queries often follow the expected structure, but column references do not match the intended schema elements.

Under full anonymization, the model continues to produce syntactically valid SQL queries that follow common patterns, such as joins, aggregations, and nested subqueries. However, these queries are often semantically incorrect, with operations applied to arbitrary columns or invalid join conditions.

Across both settings, structural query patterns are preserved, while correct schema grounding is consistently missing.

4.4 Error Cases

4.4.1 Full Anonymization

Table 5 illustrates model behavior across anonymization levels.

Level	Status	Generated SQL / Error Pattern
L_0	Correct	`SELECT Name, population, HeadOfState FROM country ORDER BY SurfaceArea DESC LIMIT 1`
L_1	Syntax Error	`SELECT` **`nm, popu, head`** `FROM country ORDER BY` **`surf`** `DESC LIMIT 1` *Lexical mismatch under abbreviation.*
L_2	Semantic Error	`SELECT` **`text_1, num_3, text_6`** `FROM table_3 ORDER BY` **`num_1`** `DESC LIMIT 1` *Type-based guessing without grounding.*
L_3	Collapse	`SELECT` **`col_7, col_2, col_13`** `FROM table_3 ORDER BY` **`col_5`** `DESC LIMIT 1` *Template preserved but column mapping is arbitrary.*

Table 5: Error patterns under full anonymization (Example ID: 790)

Several recurring patterns are observed:

- **Structural Template Persistence:** The model produces common SQL structures (e.g., `JOIN`, `GROUP BY`, `ORDER BY`) across all levels.
- **Type-Based Guessing (L_2):** Coarse type information (e.g., numeric columns) is used, but incorrect attributes are often selected.
- **Value–Schema Decoupling:** Values from the question are extracted correctly but are not matched to the appropriate columns.
- **Join Grounding Failure:** Join structures are present, but correct key relationships are not consistently identified.

4.4.2 Partial Anonymization

Table 6 shows representative examples under partial anonymization.

The following patterns are observed:

- **Lexical Rigidity (L_1):** The model relies on memorized lexical patterns, leading to mismatches under small changes.

Level	Status	Generated SQL / Error Pattern
L_0	Correct	`SELECT Name, population, HeadOfState FROM country ORDER BY SurfaceArea DESC LIMIT 1`
L_1	Syntax Error	`SELECT` **`nm, popu, head`** `FROM country ORDER BY` **`surf`** `DESC LIMIT 1` *Lexical mismatch from reliance on internal abbreviation priors.*
L_2	Semantic Error	`SELECT` **`identifier, attr_248081, attr_523ba9`** `FROM country ORDER BY` **`attr_8cd0df`** `DESC LIMIT 1` *Semantic ambiguity among related attributes.*
L_3	Collapse	`SELECT` **`col_2, col_7, col_13`** `FROM country ORDER BY` **`col_5`** `DESC LIMIT 1` *Table identified, but column mapping remains arbitrary.*

Table 6: Error patterns under partial anonymization (Example ID: 791)

- **Semantic Ambiguity (L_2):** Weak transformations lead to confusion between related attributes.
- **Relational Anchoring (L_3):** Table-level information is still used, but column selection remains unreliable.
- **Aggregation Stability:** Aggregation queries remain more stable compared to other query types.

5 Discussion

5.1 Semantic Dependence vs. Structural Reasoning

The results show that current Text-to-SQL models rely more on lexical schema semantics than on relational structure. As lexical information is progressively removed, performance drops sharply and eventually collapses, even though relational structure, including foreign key relationships, is fully preserved.

This observation suggests that relational structure alone does not provide enough signal for accurate query generation. Even when structural information is available, the model is unable to recover correct mappings between natural language queries and schema elements. In contrast, even weak lexical cues allow the model to maintain a small amount of performance, indicating that schema semantics play a central role in guiding model behavior. These results point to a heavy dependence on lexical alignment between the question and schema, rather than robust reasoning over the underlying relational structure.

5.2 Template Learning without Grounding

The qualitative results provide further insight into how this dependence shows up in model behavior. Across different anonymization settings, the model still produces syntactically valid SQL queries that follow common structural patterns, including joins, aggregations, and nested subqueries.

However, these queries are often semantically incorrect. Columns may be selected inappropriately, join conditions do not always match the correct keys, and filters are sometimes applied to unrelated attributes. This suggests that the model has learned general SQL templates, but cannot reliably ground them in the schema when semantic information is missing.

From this perspective, structural competence and semantic grounding appear to be partly decoupled. The model can generate well-formed queries, yet it struggles to assign correct roles to schema elements. This gap helps explain why execution accuracy drops sharply, even when the overall query structure appears reasonable.

Why does the model continue to generate well-formed SQL queries even when schema semantics are entirely removed (e.g., under L_3)?

One possible explanation is that the model has learned strong priors over SQL syntax and common query patterns. In this view, the model relies on memorized structural templates (e.g., `SELECT{FROM{WHERE{ORDER BY}` and continues to instantiate them even when semantic grounding is no longer possible. This is consistent with the observation that queries remain syntactically valid but semantically incorrect. An alternative answer is that the model attempts to infer structural patterns directly from the natural language question. For example, phrases such as "largest", "maximum", or "top" may trigger the generation of `ORDER BY ... DESC LIMIT 1` patterns, even without access to meaningful schema tokens.

However, the results suggest that such structural inference is limited. While the model can preserve high-level query templates, it fails to correctly bind these structures to schema elements. As a result, generated queries remain structurally plausible but semantically incorrect. This indicates that the observed behavior is better explained by template-driven generation rather than genuine structural reasoning over the database.

5.3 Role of Relational Structure

To better understand the role of relational structure, model behavior is examined under settings where foreign key relationships are preserved. Structural information can still provide some support for query generation. In many cases, the model maintains reasonable join patterns and grouping structures even under strong anonymization. This suggests that relational structure helps constrain the overall form of a query.

However, this information is not sufficient on its own. Even when foreign key relationships are available, the model often fails to assign the correct semantic roles to tables and columns. As a result, queries may be structurally valid but still produce incorrect outputs.

In practice, relational structure seems to influence *how* a query is formed (e.g., join patterns and grouping), but not *what* should be queried. The latter depends on semantic grounding. Both aspects are needed for correct query generation, but current models appear to rely more heavily on semantic cues.

5.4 Sensitivity to Anonymization Design

To examine whether the results depend on the anonymization design, two schemes are compared: full anonymization and partial anonymization.

Full anonymization removes all lexical information from table and column names, leaving only relational structure. Partial anonymization, in contrast, retains some weak lexical cues, such as simplified table or column names.

The overall pattern is similar across both schemes. As lexical information is reduced, performance drops quickly. Under partial anonymization, execution accuracy decreases from over 90% to around 5%, while under full anonymization it falls to zero at higher levels.

The main difference is the presence of residual semantic signals. With partial anonymization, weak lexical cues still provide limited guidance, especially for aggregation queries. Under full anonymization, this signal is removed completely, and performance fails.

A closer look by query type shows the same trend. Under partial anonymization, aggregation queries are the only category with non-zero performance, while others fall to zero. This may be because some aggregation operations, such as `COUNT(*)`, can be inferred directly from the question without precise schema alignment.

Figure 2 shows that this pattern is consistent across query types. Partial anonymization retains small but non-zero performance in a few cases, while full anonymization leads to complete failure.

Overall, performance decreases gradually as lexical information is removed. Even weak semantic cues help to some extent, but they are not enough to support reliable query generation on their own.

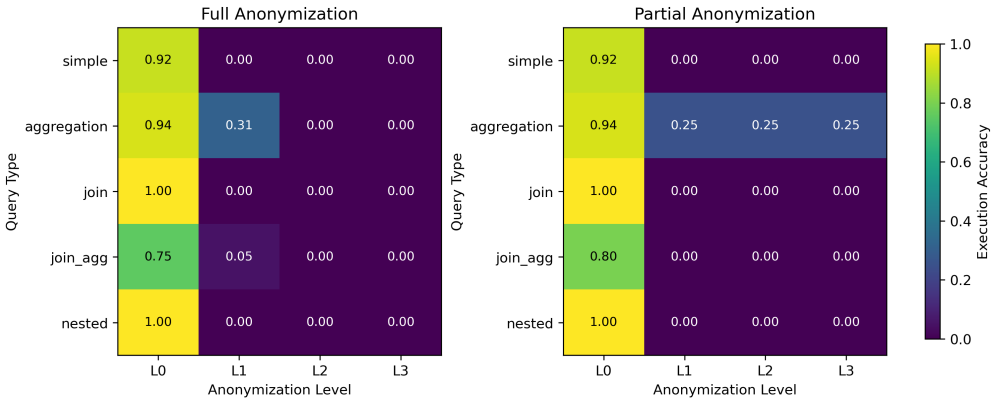

Figure 2: Execution accuracy by query type under different anonymization schemes. The left panel shows full anonymization, where only relational structure is preserved. The right panel shows partial anonymization, where limited lexical information remains. Performance degrades in both settings, but partial anonymization retains small gains due to residual semantics.

5.5 Implications

These findings have several implications for how Text-to-SQL systems are evaluated and developed. In particular, high execution accuracy on standard benchmarks does not necessarily reflect robust structural reasoning, and may instead depend heavily on surface-level lexical alignment.

This raises concerns about generalization in settings where schema semantics are noisy, ambiguous, or unavailable. In such situations, models may fail even when the underlying relational structure is preserved.

The results also point to a limitation of current LLM-based approaches. Strong performance on structured tasks does not by itself guarantee that the model is making meaningful use of the underlying structure. This suggests that improving schema grounding, rather than relying primarily on implicit lexical matching, is essential for building more robust Text-to-SQL systems.

5.6 Limitations

There are three main limitations. The experiments are conducted on a relatively small subset of the Spider dataset (80 examples), which may limit the generalizability of the findings. The study evaluates a single model under a fixed prompting setup. Different models or prompting strategies may exhibit different sensitivities to schema anonymization. While the anonymization framework isolates the role of schema semantics, it does not account for other factors such as database content distribution or alternative schema representations, which may also influence model behavior.

6 Conclusion

This study investigates the relative roles of schema semantics and relational structure in Text-to-SQL models through a controlled schema anonymization framework. By progressively removing lexical information while preserving relational structure, we evaluate model performance across multiple query types on a subset of the Spider benchmark.

Our results show that performance depends strongly on schema semantics. While models achieve high accuracy with the original schema, performance degrades sharply under partial anonymization and collapses under full anonymization. This pattern is consistent across query types, indicating that relational structure alone is insufficient for accurate query generation.

Overall, the findings highlight a gap between structural competence and semantic grounding. Improving the integration of these two aspects remains an important direction for building more robust Text-to-SQL systems.

REFERENCES

Zhoujun Cheng, Tianyi Liu, and Jian Wang. Binding language models in symbolic languages. In *ICLR*, 2023.

Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. Towards complex Text-to-SQL in cross-domain database with intermediate representation. In *ACL*, 2019.

Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of Text-to-SQL with self-correction. In *NeurIPS*, 2023.

Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. Rat-sql: Relation-aware schema encoding and linking for Text-to-SQL parsers. In *ACL*, 2020.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and Text-to-SQL task. In *EMNLP*, 2018.